

Course Builder

Three endpoints. **Plan** turns a topic into a course. **Build** queues every lesson as a video. **Status** reports progress. Everything is read-only from your side except `build`, which starts work but writes nothing into your system.

- BASE URL <https://content-queen-production.up.railway.app>
- TRY IT </demo/course-builder> — the working planner. No credential needed: just type a topic. Rate limited to 5 plans an hour.
- DISCOVER [GET /api/v1](/api/v1) — public, machine-readable list of every route on this service.

Authentication

The same bearer token as the checkpoint API, required on every `/api/v1` route. Server to server — never from a browser, or you ship the token to every learner. (The `/demo` planner is deliberately open and rate limited, so you can try the idea before wiring anything.)

CREDENTIAL · TREAT AS A PASSWORD

HEADER **Authorization: Bearer <token>**

TOKEN <sent separately — never written into a document>

Same token, same service as the in-video question API. **Do not put it in browser code** or commit it. If it leaks, tell Aroma; replacing it is one command and changes nothing else.

The three endpoints

ENDPOINT	DOES	COSTS
POST <code>/api/v1/courses/plan</code>	Topic in, full course plan out. Idempotent in the sense that it never changes anything — call it as often as you like.	~60–90s. One model call. No video spend.
POST <code>/api/v1/courses/build</code>	Queues every lesson in a plan as a video. Returns immediately with a <code>courseId</code> ; the work happens over hours.	~\$1.50 and ~30 min PER LESSON.
GET <code>/api/v1/courses/{courseId}</code>	Progress of a queued course.	Free. Poll every 30–60s.

THE MISTAKE THIS DESIGN EXISTS TO PREVENT

Planning is free and takes a minute. Building costs real money and takes hours. **Never chain plan straight into build.** A human must see the cost and approve it. Our API enforces this — `build` refuses unless you echo back the exact lesson count — but the meaningful guard is the screen you put in front of the user.

POST /api/v1/courses/plan

One required field. Everything else is optional — and if omitted, the planner picks defaults and lists them in **assumptions**.

REQUEST

```
curl -X POST https://content-queen-production.up.railway.app/api/v1/courses/plan \
-H "Authorization: Bearer $CONTENT_API_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "topic":      "Using WhatsApp Business to take orders without losing any", // required
  "duration":   "about 15 minutes",    // optional, free text
  "audience":  "shopkeepers with a smartphone", // optional
  "description": "focus on catalogue and labels" // optional
}'
```

Response (200) — trimmed to one lesson; a real plan has 4–8

RESPONSE • APPLICATION/JSON

```
{
  "title":  "WhatsApp Business: Take Orders Without Losing Any",
  "summary": "One paragraph for your course catalogue.",
  "audience": "Small shopkeepers in Pakistan who already take orders on WhatsApp",
  "protagonist_scenario": "Ali runs a dairy shop in Gujranwala... lost eleven orders last month",
  "modules": [
    {
      "title":  "Seeing where the orders leak",
      "summary": "...",
      "lessons": [
        {
          "title":      "Spot the three places an order goes missing",
          "slo":        "Given a chat thread where an order was lost, the learner can name which leak caused it",
          "difficulty": "novice", // novice | intermediate | advanced | expert
          "brief":      "The step of Ali's story this lesson covers – becomes the script brief",
          "question": {
            "stem":      "A customer messages at 9pm... Which leak is this?",
            "options":    ["Read but not recorded", "Incomplete details", "No update sent", "Cancelled"],
            "correctIndex": 0,
            "explanation": "Why the common wrong choices are wrong, not just the right answer"
          }
        }
      ]
    }
  ],
  "assumptions": ["No audience was given, so..."], // SHOW THESE TO THE USER
  "request": { /* what you sent, echoed back */ },
  "estimate": {
    "lessons":      8,
    "estimatedCostUsd": 12, // show this before any build button
    "estimatedBuildMinutes": 240,
    "estimatedWatchMinutes": 20
  },
  "plannedAt": "2026-09-11T12:04:11.902Z"
}
```

FIELD	USE IT FOR
<code>modules[].lessons[]</code>	One lesson = one video. This is the unit everything else counts in. Build your course structure from it directly.
<code>lesson.question</code>	The in-video question for that lesson. Same shape as the checkpoint API you already render — you can show it in the course outline before the video exists.
<code>assumptions</code>	Show these. They are how a user learns the planner guessed something, and the cheapest way to get a better plan is to let them correct one and re-plan.
<code>estimate</code>	Put on screen before any build control. It is the only thing standing between a curious click and \$12.

POST /api/v1/courses/build

Send back the plan you got, the series it belongs to, and a confirmation of the lesson count.

REQUEST
<pre>{ "plan": { /* the whole object from /courses/plan, unmodified */ }, "series": "whatsapp-orders", // required; lowercase a-z 0-9 . _ - "confirmLessons": 8 // must equal plan's lesson count }</pre>

Response (202 Accepted)

RESPONSE
<pre>{ "courseId": "course-m1k2j3h4", // store this; it is how you poll "series": "whatsapp-orders", "queued": 8, "rejected": [], // lessons that could not queue, with reasons "items": ["whatsapp-orders/spot-the-three-places-an-order-goes-missing", "..."], "status": "https://.../api/v1/courses/course-m1k2j3h4" }</pre>

Errors you must handle

STATUS	MEANING AND WHAT TO DO
409 <code>confirmation_required</code>	Not an error — the guard working. The body carries the real lesson count and the estimate. Show them, get a human yes, then resend with <code>confirmLessons</code> set to that number.
400 <code>bad_request</code>	Missing <code>plan</code> , or a <code>series</code> that is absent or not lowercase-slug shaped. The series is never guessed: a wrong one files an entire course in the wrong place and is only noticed after the money is spent.
401 <code>unauthorized</code>	Bad or missing bearer token.
503 <code>api_not_configured</code>	The service has no token configured and is failing closed. Ours to fix — retry later.
500 <code>plan_failed</code>	Planning failed upstream. Safe to retry once; nothing was spent.
Partial queue	A 202 with a non-empty <code>rejected</code> array means some lessons queued and some did not — usually a duplicate slug. Surface it; do not treat the course as fully started.

GET /api/v1/courses/{courseId}

RESPONSE
<pre>{ "courseId": "course-m1k2j3h4", "lessons": 8, "done": 3, "failed": 0, "inProgress": 5, "items": [{ "id": "whatsapp-orders/spot-the-three-places...", "topic": "...", "status": "done", "module": 1 }] }</pre>

Poll every 30–60 seconds. A finished lesson's video and its in-video question then come from the checkpoint API you already use: `GET /api/v1/videos/{videoId}/checkpoints`, where `videoId` is the part of the item id after the slash.

Limits that will affect your UI

LIMIT	DESIGN AROUND IT BY
Planning takes 60–90 seconds	A real progress state, not a spinner that looks hung. Our own prototype says "this takes about a minute".
Building takes hours	Treat a course build as a background job with a progress list. Never block a request on it.
We store no plans	Persist the plan on your side as soon as you show it. If the user navigates away, it is gone — we kept nothing.
A redeploy costs one lesson	The course itself survives — the queue is held on a mounted volume. The lesson that was mid-build does not: its render files are stripped from our image and do not come back, so it is parked as blocked with interrupted: true and waits for a person. Approving it <i>rebuilds</i> that lesson. Show per-lesson status rather than one percentage, so a stall is visible.
Videos render one at a time	Two courses started together take twice as long. Consider allowing one build at a time per user.
Lessons can fail QA and be remade	failed in the status response is real. Decide whether a course appears when partly ready.

Approving, rejecting, and the states a lesson moves through

A course pauses at every lesson. These are the two calls that move it, and the only states you will see in `items[].status`.

CALL	WHAT IT DOES
POST <code>/api/v1/courses/{courseId}/lessons/{series}/{slug}/approve</code>	Publishes that lesson and releases the next one. The lesson id contains a slash; send it unescaped. <code>by</code> is recorded against your tenant , taken from your credential rather than the field, exactly as on <code>/demo/make-video/:id/approve</code> .
POST <code>.../reject</code>	Terminal. The course stops and nothing after that lesson is built. Lessons already approved stay published — a reject does not unpublish them. There is no un-reject; re-plan to start again.

STATUS	MEANS
<code>queued</code>	Waiting. Nothing spent on it yet.
<code>claimed</code>	Being built right now. Recorded before any spend, so an interrupted build is always distinguishable from one that never started.
<code>blocked</code>	Waiting for a person. This is the pause. Check <code>awaitingApproval</code> in the status response for which lesson and why. An <code>interrupted: true</code> here means a restart caught it mid-build and approving it rebuilds rather than publishes.
<code>done</code>	Built, approved and published.
<code>failed</code>	Failed QA, or rejected by a human.

Five smaller answers

Error shape	Always { "error": "<slug>", "message": "<prose>" }, with extra fields where they help — 409 carries lessons and estimate; the 404 carries durability.
Idempotency-Key	Not on /courses/build — only on /produce. Build is guarded by the confirmLessons echo plus duplicate-slug rejection: a repeat returns 202 with the duplicates in rejected, not a second course. Ask and we will add the key.
callbackUrl	Not for courses yet — it is accepted on POST /demo/make-video only. Poll GET /courses/{courseId} every 30–60s. Ask and it is the same signed-webhook path.
Rate limits	Keyed on your tenant, not your IP — one egress address does not throttle your whole university. Every 429 carries Retry-After and X-RateLimit-*.
What is charged, when	Per lesson, as each is released — never the whole course up front. A course stopped after lesson two has cost two lessons.

How it fits what you have already built

YOU ALREADY HAVE	IT WORKS UNCHANGED
The in-video question popup	Courses built this way produce videos with the same checkpoints, from the same endpoint. Nothing to change.
The bearer token	Same token, same service, same header.
Your course/lesson tables	Map modules[].lessons[] onto them. We deliberately do not impose a schema — tell us yours and we will shape the plan to fit rather than the other way round.

ONE THING THAT IS GENUINELY UNFINISHED

Plans cannot be edited. A user accepts a plan or re-plans with a better description. If editing matters to you, tell us — it changes the contract, because an edited plan has to come back to us.

Build durability was the second item here, and it is now done. The queue moved onto the job store, so it follows a Railway volume and a course outlives a redeploy. GET /api/v1/health reports courses.durability, and the 404 body on an unknown courseId reports it too, so you never have to guess whether a course was lost or never existed. Until the volume is attached it reads "container" — survives a restart, not a redeploy — and says so rather than claiming otherwise.